Programátorská dokumentace - Game Reviews Platforma

Úvod

Tato dokumentace je určena pro programátory, kteří chtějí pochopit vnitřní fungování aplikace, její architekturu, algoritmy a strukturu zdrojového kódu. Popisuje technické detaily implementace, které nejsou viditelné z uživatelského rozhraní.

Obsah

1. Architektura aplikace
2. Struktura projektu
3. Routing systém
4. Middleware systém
5. Validace dat
6. Zpracování obrázků
7. Databázové modely
8. Session management
9. CSRF ochrana
10. Paginace
11. AJAX endpointy
12. Bezpečnostní aspekty

Architektura aplikace

Aplikace je postavena na **MVC (Model-View-Controller)** architektuře bez použití frameworku. Všechno je implementováno vlastními třídami a funkcemi.

Tok požadavku

1. **Entry point** (`index.php`): Načte router a předá mu řízení
2. **Router** (`app/includes/router.php`): Parsuje URL, najde odpovídající route
3. **Middleware Stack**: Spustí middleware v pořadí (autentizace, CSRF, validace)
4. **Controller**: Zpracuje požadavek, zavolá model pro data
5. **Model**: Provede databázové operace
6. **View**: Vykreslí HTML šablonu s daty
7. **Response**: Vrátí HTML stránku uživateli

Struktura projektu

```
game-reviews/
├── app/
│   ├── controllers/        # Controller funkce (zpracování požadavků)
```

```
│   ├── models/             # Databázové operace a logika
│   ├── views/              # HTML šablony
│   └── includes/
│       ├── config.php       # Konfigurace, DB připojení, session
│       ├── router.php       # Routing systém
│       ├── middlewares/     # Middleware třídy
│       └── services/        # Pomocné služby (validace, CSRF, paginace)
├── public/
│   ├── assets/              # Statické soubory (ikony)
│   ├── css/                 # Styly
│   ├── js/                  # JavaScript (validace, AJAX, carousel)
│   └── uploads/             # Nahrané obrázky
├── docs/                    # Dokumentace
└── index.php               # Entry point
```

Controllers

Controllers jsou funkce, ne třídy. Každý controller soubor obsahuje funkce pro různé akce: - `HomeController.php` - hlavní stránka - `GamesController.php` - správa her a recenzí - `AuthController.php` - přihlášení, registrace, odhlášení - `UserController.php` - uživatelské profily - `AdminController.php` - administrátorský panel - `UtilController.php` - utility stránky (404, 403)

Models

Modely obsahují funkce pro práci s databází: - `GameModel.php` - operace s hrami - `ReviewModel.php` - operace s recenzemi a reakcemi - `UserModel.php` - operace s uživateli - `AuthModel.php` - autentizace a registrace - `TagsModel.php` - tagy (žánry, platformy)

Views

Views jsou PHP soubory s HTML a minimální logikou: - Používají `renderView()` pro vnořené šablony - Data se předávají jako proměnné přes `extract()` - Částečné šablony v `partials/` pro opakující se komponenty

---

Routing systém

Routing je implementován v `app/includes/router.php` jako jednoduchý asociativní array.

Jak to funguje

1. **Parsování URL**:

   ```php
   $uri = parse_url($_SERVER['REQUEST_URI'], PHP_URL_PATH);
   $base = parse_url(APP_BASE, PHP_URL_PATH);
   $path = trim(str_replace($base, '', $uri), '/');
   ```

   Odstraní base URL a získá čistou cestu (např. `games/create`).

2. **Nalezení route**:

```
if (isset($routes[$path])) {
    $route = $routes[$path];
}
```

3. **Parsování controlleru**:

```
[$controllerName, $method] = explode('@', $route['controller']);
// Např. 'GamesController@showGamesPage' -> ['GamesController',
'showGamesPage']
```

4. **Načtení controlleru**: Dynamicky se načte soubor a zkontroluje existence funkce.

5. **Spuštění middleware stacku**: Všechny middleware se spustí před controllerem.

Formát route

```
'cesta' => [
    'controller' => 'ControllerName@methodName',
    'middleware' => [
        new AuthMiddleware('user'),
        new CsrfMiddleware('/redirect-url', 'session-key'),
        new ValidationMiddleware([...rules...], '/redirect-url', 'session-
key')
    ]
]
```

Příklad:

```
'games/add' => [
    'controller' => 'GamesController@submitGame',
    'middleware' => [
        new AuthMiddleware('user'),
        new CsrfMiddleware('/games/create', 'game'),
        new ValidationMiddleware([
            'title' => [['required'], ['string'], ['min', 1], ['max', 255]],
            'cover_image' => [['required'], ['image'], ['image_max_size', 5 *
1024 * 1024]]
        ], '/games/create', 'game')
    ]
]
```

Query parametry

Query parametry (např. `?id=5`) se zpracovávají v controlleru pomocí `$_GET`. Router je ignoruje - řeší pouze cestu.

Middleware systém

Middleware jsou třídy implementující `MiddlewareInterface` s metodou `handle(callable $next)`.

MiddlewareStack

`MiddlewareStack` vytváří řetězec middleware pomocí rekurzivních closure:

```
private function createNext(int $index, callable $controller): callable {
    return function() use ($index, $controller) {
        if ($index < count($this->middlewares)) {
            $middleware = $this->middlewares[$index];
            $middleware->handle($this->createNext($index + 1, $controller));
        } else {
            $controller();
        }
    };
}
```

Jak to funguje: 1. Vytvoří se closure pro každý middleware 2. Každý middleware dostane `$next` - callback na další middleware/controller 3. Middleware může buď zavolat `$next()` (pokračovat) nebo přesměrovat (přerušit)

AuthMiddleware

Kontroluje autentizaci a autorizaci:

```
switch ($this->type) {
    case 'guest': requireGuest(); break;  // Musí být odhlášen
    case 'user': requireUser(); break;    // Musí být přihlášen
    case 'admin': requireAdmin(); break; // Musí být admin
}
```

Pokud kontrola selže, middleware přesměruje a ukončí execution (nevolá `$next()`).

CsrfMiddleware

Kontroluje CSRF token pouze pro POST požadavky:

1. Získá token z `$_POST['csrf_token']`
2. Ověří pomocí `validateCsrfToken()`
3. Pokud selže, uloží chybu do session a přesměruje zpět
4. Zachová query parametry (např. `?id=5`)

Důležité: Token se po validaci smaže z session (single-use token).

ValidationMiddleware

Validuje formulářová data před zpracováním:

1. Vytvoří `Validator` instanci s `$_POST` a `$_FILES`
2. Spustí validaci podle pravidel
3. Pokud jsou chyby:
    - Uloží chyby do session (`{key}_errors`)

  - Uloží staré hodnoty (`{key}_old`) - pro PRG pattern
  - Přesměruje zpět na formulář
4. Pokud je vše ok, pokračuje k controlleru

Validace dat

Validace probíhá na **dvou úrovních**: frontend (JavaScript) a backend (PHP).

Backend validace (Validator třída)

`Validator` třída v `app/includes/services/Validator.php` validuje data podle pravidel:

```
$validator = new Validator($_POST, $_FILES);
$errors = $validator->validate([
    'username' => [['required'], ['username'], ['max', 50]],
    'email' => [['required'], ['email']],
    'avatar' => [['image'], ['image_max_size', 2 * 1024 * 1024]]
]);
```

Dostupná pravidla: - `required` - pole musí být vyplněno - `string` - hodnota musí být string - `email` - validní email formát - `min/max` - délka textu - `username` - formát uživatelského jména (regex) - `password` - síla hesla (velké písmeno, číslo, speciální znak) - `confirmed` - potvrzení hesla - `image` - validní obrázek (MIME type + přípona) - `image_max_size` - maximální velikost souboru - `year` - rok v rozsahu - `rating` - hodnocení v rozsahu 1-10 - `array_not_empty` - pole musí obsahovat alespoň jeden prvek

Validace obrázků:

```
case 'image':
    $allowedMimes = ['image/jpeg', 'image/png', 'image/webp'];
    $allowedExtensions = ['jpg', 'jpeg', 'png', 'webp'];

    $mime = mime_content_type($file['tmp_name']);
    $extension = strtolower(pathinfo($file['name'], PATHINFO_EXTENSION));

    if (!in_array($mime, $allowedMimes) || !in_array($extension,
$allowedExtensions)) {
        $this->addError($field, 'Podporované formáty: JPG, PNG, WEBP.');
    }
```

Kontroluje jak MIME type (skutečný typ souboru), tak příponu (jako záložní metoda).

Frontend validace (ClientValidator třída)

JavaScript validace v `public/js/validation.js` duplikuje backend pravidla pro okamžitou zpětnou vazbu:

```
class ClientValidator {
    validate() {
```

```
        // Extrahuje data z FormData
        // Aplikuje stejná pravidla jako backend
        // Zobrazí chyby před odesláním
    }
}
```

Inicializace: Formuláře mají `data-validation-rules` atribut s JSON pravidly:

```
<form data-validation-rules='{"username":[["required"],["username"]]}'>
```

Důležité: Frontend validace je pouze pro UX - backend validace je finální kontrola.

Zpracování obrázků

Obrázky se zpracovávají pomocí GD knihovny a ukládají se ve formátu WebP.

Algoritmus resize a crop

Funkce `imageResizeWebp()` v `app/includes/services/helpers.php`:

1. **Načtení obrázku** podle MIME typu:

   ```
   switch ($mime) {
       case 'image/jpeg': $src = imagecreatefromjpeg($srcPath); break;
       case 'image/png':  $src = imagecreatefrompng($srcPath);  break;
       case 'image/webp': $src = imagecreatefromwebp($srcPath); break;
   }
   ```

2. **Výpočet poměru** pro zachování aspect ratio:

   ```
   $ratio = max($newW / $origW, $newH / $origH);
   $resizeW = (int)($origW * $ratio);
   $resizeH = (int)($origH * $ratio);
   ```

   Používá `max()` aby obrázek pokryl celou cílovou plochu (možná bude větší).

3. **Resize** na dočasnou velikost:

   ```
   $tmp = imagecreatetruecolor($resizeW, $resizeH);
   imagecopyresampled($tmp, $src, 0, 0, 0, 0, $resizeW, $resizeH, $origW,
   $origH);
   ```

4. **Crop** na finální velikost (centrovaný):

   ```
   $cropX = (int)(($resizeW - $newW) / 2);
   $cropY = (int)(($resizeH - $newH) / 2);
   imagecopy($dst, $tmp, 0, 0, $cropX, $cropY, $newW, $newH);
   ```

5. **Uložení jako WebP** s kvalitou 85%:

```
    imagewebp($dst, $destPath, 85);
```

Výsledek: Obrázek je vždy přesně požadované velikosti, oříznutý z centra.

Obálky her

`uploadGameCovers()` vytváří tři verze obrázku: - `cover_full.webp` - 600x900px (detail hry) - `cover_thumb_vertical.webp` - 200x300px (seznamy) - `cover_thumb_horizontal.webp` - 300x170px (karusely)

Ukládají se do `public/uploads/games/YYYY-MM-DD_ID/`.

Avatary

`uploadAvatar()` vytváří jednu verzi: - 200x200px WebP - Ukládá se do `public/uploads/avatars/` s unikátním názvem

---

Databázové modely

Modely používají PDO s prepared statements pro ochranu proti SQL injection.

Typické dotazy

Paginace s JOIN:

```
SELECT
    g.id, g.title, g.description,
    COALESCE(AVG(r.rating), 0) AS average_rating,
    COUNT(DISTINCT r.id) AS review_count,
    GROUP_CONCAT(DISTINCT CASE WHEN t.type = 'genre' THEN t.name END
SEPARATOR '|') AS genres
FROM games g
LEFT JOIN reviews r ON g.id = r.game_id
LEFT JOIN game_tags gt ON g.id = gt.game_id
LEFT JOIN tags t ON gt.tag_id = t.id
WHERE g.status = :status
GROUP BY g.id, g.title, g.description
ORDER BY average_rating DESC
LIMIT :limit OFFSET :offset
```

Důležité techniky: - `COALESCE()` pro výchozí hodnoty (0 místo NULL) - `GROUP_CONCAT()` pro agregaci tagů do stringu - `CASE WHEN` pro filtrování tagů podle typu - `COUNT(DISTINCT)` pro správné počítání recenzí

Zpracování dat

Po dotazu se data často zpracovávají:

```
function processGamesData(array $games): array {
    foreach ($games as &$game) {
```

```
        // Dekódování JSON (obálky)
        if ($game['covers']) {
            $game['covers'] = json_decode($game['covers'], true);
        }

        // Rozdělení stringu na pole (tagy)
        if ($game['genres']) {
            $game['genres'] = explode('|', $game['genres']);
        }
    }
    return $games;
}
```

Session management

Session se spouští v `config.php` s bezpečnostními nastaveními:

```
session_set_cookie_params([
    'lifetime' => 604800,        // 7 dní
    'path' => '/',
    'domain' => '',
    'secure' => isset($_SERVER['HTTPS']) && $_SERVER['HTTPS'] === 'on',
    'httponly' => true,          // JavaScript nemůže přistupovat
    'samesite' => 'Strict'       // Ochrana proti CSRF
]);
session_start();
```

Uživatelská data

Po přihlášení se ukládají do `$_SESSION['user']`:

```
$_SESSION['user'] = [
    'id' => $user['id'],
    'username' => $user['username'],
    'role' => $user['role'],
    'is_blocked' => (bool)($user['is_blocked'] ?? false)
];
```

Flash messages

Flash messages se ukládají do session a automaticky se mažou po zobrazení:

```
// Uložení
$_SESSION['review_errors'] = $errors;
$_SESSION['review_old'] = $oldData;

// Zobrazení (v view)
$errors = getFlash('review_errors') ?? [];
$old = getFlash('review_old') ?? [];

// getFlash() automaticky smaže hodnotu z session
```

PRG pattern (Post-Redirect-Get): Po POST se data uloží do session, přesměruje se na GET, data se zobrazí a smažou.

CSRF ochrana

CSRF token se generuje při každém načtení formuláře a validuje se při odeslání.

Generování tokenu

```
function generateCsrfToken(): string {
    if (!isset($_SESSION['csrf_token'])) {
        $_SESSION['csrf_token'] = bin2hex(random_bytes(32)); // 64 znaků hex
    }
    return $_SESSION['csrf_token'];
}
```

Token se generuje pomocí `random_bytes()` (kryptograficky bezpečné) a ukládá se do session.

Validace tokenu

```
function validateCsrfToken(string $token): bool {
    if (!isset($_SESSION['csrf_token'])) {
        return false;
    }

    $isValid = hash_equals($_SESSION['csrf_token'], $token);

    if ($isValid) {
        unset($_SESSION['csrf_token']); // Single-use token
    }

    return $isValid;
}
```

Důležité: - Token se smaže po validaci (single-use) - Pokud token není v session, validace selže

Zobrazení v formuláři

```
function csrfField(): string {
    $token = generateCsrfToken();
    return '<input type="hidden" name="csrf_token" value="' .
htmlspecialchars($token) . '">';
}
```

V šabloně: `<?= csrfField() ?>`

Paginace

Paginace je implementována jako služba s ukládáním stavu do session. Hlavní výhoda tohoto přístupu je, že po POST redirectu (PRG pattern) nebo při navigaci jinam a zase zpatky se zachovají původní parametry paginace (např. řazení, stránka), takže uživatel se vrátí na stejné místo v seznamu.

Když uživatel odešle formulář (např. přidá hru), nebo přes navigaci půjde na jiný tab a pak zase zpátky, aplikace ho přesměruje na seznam her. Bez ukládání stavu do session by se ztratily parametry jako `sort` nebo `page`, takže by se uživatel vrátil na první stránku s výchozím řazením. Session řeší tento problém.

Ukládání stavu

Funkce `updatePaginationState()` aktualizuje stav paginace z aktuálního požadavku:

```
function updatePaginationState(string $key, array $allowedParams = ['page',
'sort']): void {
    $savedParams = getPaginationState($key);
    $params = [];

    foreach ($allowedParams as $param) {
        if (isset($_GET[$param])) {
            // Pokud je parametr v URL, použije se
            $params[$param] = $_GET[$param];
        } elseif (isset($savedParams[$param])) {
            // Pokud není v URL, ale je v session, použije se z session
            // ALE: page se nikdy nepoužije z session pro update (když není
page param v getu, znamená že jsme na 1 stránce a chceme tam být úmyslně,
takže chování, že neuložíme page z session je správné)
            if ($param === 'page') {
                continue;
            }
            $params[$param] = $savedParams[$param];
        }
    }

    savePaginationState($key, $params);
}
```

Důležité chování: - Pokud je parametr v `$_GET`, použije se z URL - Pokud není v URL, ale je v session, použije se z session (kromě `page`) - `page` se nikdy nepoužije z session - vždy se resetuje, pokud není v URL - To zajišťuje, že při přímém přístupu na URL bez `page` se zobrazí první stránka

Příklad použití v controlleru:

```
function showGamesPage(PDO $pdo): void {
    // Aktualizuje stav paginace z $_GET
    updatePaginationState('games');
```

```
    // Získá aktuální stránku (z $_GET nebo výchozí 1)
    $page = max(1, (int)($_GET['page'] ?? 1));
    $sort = $_GET['sort'] ?? 'rating_desc';

    // Zavolá model s paginací
    $result = getGamesPaginated($pdo, 'active', $page, 12, $sort);
}
```

Klíče paginace

Každá stránka má svůj vlastní klíč v session: - `'games'` - seznam her - `'admin'` - administrátorský panel (pouze `page`, ne `sort`) - `'admin_pending'` - hry čekající na schválení - `'user_5'` - profil uživatele s ID 5 (dynamický klíč)

Dynamické klíče: Pro uživatelské profily se používá `'user_' . $userId`, aby každý profil měl nezávislý stav paginace.

Sestavení URL

Funkce `buildPaginationUrl()` sestaví URL s parametry paginace:

```
function buildPaginationUrl(string $baseUrl, string $key, array
$overrideParams = []): string {
    // Získá uložené parametry z session
    $savedParams = getPaginationState($key);

    // Merge: nejdřív uložené, pak override (override má přednost, ovverride
se zkrátká použije při použití paginace na FE např. změna stránky)
    $params = array_merge($savedParams, $overrideParams);

    // Parsuje existující query parametry z baseUrl (např. /user?id=11)
    $urlParts = parse_url($baseUrl);
    $basePath = $urlParts['path'] ?? $baseUrl;
    $existingParams = [];

    if (isset($urlParts['query'])) {
        parse_str($urlParts['query'], $existingParams);
    }

    // Merge existujících parametrů s paginací
    $params = array_merge($existingParams, $params);

    // Odstraní page=1 (výchozí hodnota, není potřeba v URL)
    if (isset($params['page']) && $params['page'] == 1) {
        unset($params['page']);
    }

    // Pokud nejsou žádné parametry, vrátí čistou URL
    if (empty($params)) {
```

```
        return $basePath;
    }


    // Sestaví query string
    return $basePath . '?' . http_build_query($params);
}
```

Příklad použití:

```
// Uložený stav: ['sort' => 'date_desc', 'page' => 3]
// Override: ['page' => 2]
// Výsledek: /games?sort=date_desc&page=2

$url = buildPaginationUrl('/games', 'games', ['page' => 2]);
```

Složitější příklad (s existujícími parametry):

```
// baseUrl: /user?id=5
// Uložený stav: ['sort' => 'rating_desc']
// Override: ['page' => 2]
// Výsledek: /user?id=5&sort=rating_desc&page=2

$url = buildPaginationUrl('/user?id=5', 'user_5', ['page' => 2]);
```

Důležité: - `overrideParams` mají vždy přednost před uloženým stavem - Existující query parametry z `baseUrl` se zachovají - `page=1` se automaticky odstraní (čistší URL)

Použití po POST redirectu

Po úspěšném odeslání formuláře se přesměruje s původními parametry:

```
function submitGame(PDO $pdo): void {
    // ... zpracování formuláře ...

    // Sestaví URL s původními parametry paginace
    $redirectUrl = buildPaginationUrl('/games', 'games');
    $successMessage = 'Hra byla úspěšně přidána.';
    redirectWithSuccess($redirectUrl, $successMessage);
}
```

Uživatel se tak vrátí na stejnou stránku se stejným řazením, jaké měl před odesláním formuláře.

Zobrazení v view

V šabloně se používá `buildPaginationUrl()` pro odkazy na stránky:

```
// V partials/pagination.php
<a href="<?= APP_BASE ?><?= buildPaginationUrl($baseUrl, $key, ['page' =>
$currentPage - 1]) ?>">
```

```
    Předchozí
</a>

<a href="<?= APP_BASE ?><?= buildPaginationUrl($baseUrl, $key, ['page' =>
$currentPage + 1]) ?>">
    Další
</a>
```

Příklad: Pokud je aktuální stav `['sort' => 'date_desc', 'page' => 3]`: - Odkaz na předchozí: `/games?sort=date_desc&page=2` - Odkaz na další: `/games?sort=date_desc&page=4`

Řazení se zachová při přechodu mezi stránkami.

Databázová paginace

V modelech se používá standardní SQL paginace:

```
function getGamesPaginated(PDO $pdo, string $status, int $page, int $perPage,
string $sort): array {
    // Normalizace stránky (minimálně 1)
    $page = max(1, $page);
    $offset = ($page - 1) * $perPage;

    // Získání celkového počtu
    $countStmt = $pdo->prepare("SELECT COUNT(DISTINCT g.id) as total FROM
games g WHERE g.status = :status");
    $countStmt->execute(['status' => $status]);
    $total = (int)$countStmt->fetch()['total'];
    $totalPages = max(1, (int)ceil($total / $perPage));

    // Dotaz s LIMIT a OFFSET
    $stmt = $pdo->prepare("
        SELECT g.*, ...
        FROM games g
        WHERE g.status = :status
        ORDER BY ...
        LIMIT :limit OFFSET :offset
    ");

    $stmt->bindValue(':status', $status);
    $stmt->bindValue(':limit', $perPage, PDO::PARAM_INT);
    $stmt->bindValue(':offset', $offset, PDO::PARAM_INT);
    $stmt->execute();

    return [
        'games' => $stmt->fetchAll(),
        'total' => $total,
        'pages' => $totalPages,
        'current_page' => $page,
```

```
        'per_page' => $perPage
    ];
}
```

Důležité: - `$offset` se počítá jako `($page - 1) * $perPage` - `LIMIT` a `OFFSET` se bindují jako `PDO::PARAM_INT` (bezpečnost) - Vrací se i metadata (celkový počet, počet stránek) pro zobrazení paginace

Edge cases

Prázdné parametry: Pokud nejsou žádné parametry, `buildPaginationUrl()` vrátí čistou URL bez query stringu.

Page = 1: Parametr `page=1` se automaticky odstraní z URL (výchozí hodnota).

```
$page = max(1, min($page, $totalPages)); // Omezí na rozsah 1..totalPages
```

Změna řazení: Při změně řazení (např. z `rating_desc` na `date_desc`) se `page` resetuje na 1, protože nové řazení má jiný pořádek záznamů.

AJAX endpointy

Některé akce probíhají přes AJAX.

Formát odpovědi

Všechny AJAX endpointy vracejí JSON:

```
header('Content-Type: application/json');
echo json_encode([
    'success' => true,
    'message' => 'Recenze byla smazána.',
    'data' => [...]
]);
exit;
```

Příklady endpointů

Smazání recenze (`/api/review/delete`):

```
function deleteReview(PDO $pdo): void {
    header('Content-Type: application/json');

    $reviewId = (int)($_POST['review_id'] ?? 0);
    $userId = $_SESSION['user']['id'];

    $success = doDeleteReview($pdo, $reviewId, $userId, isAdmin());

    echo json_encode(['success' => $success, 'message' => ...]);
```

```
    exit;
}
```

Reakce na recenzi (`/api/review/reaction`):

```
function toggleReaction(PDO $pdo): void {
    $reviewId = (int)($_POST['review_id'] ?? 0);
    $reaction = $_POST['reaction'] ?? ''; // 'like' nebo 'dislike'

    $result = toggleReviewReaction($pdo, $reviewId, $userId, $reaction);
    $counts = getReviewReactionCounts($pdo, $reviewId);

    echo json_encode([
        'success' => true,
        'action' => $result['action'], // 'added', 'removed', 'changed'
        'reaction' => $result['reaction'],
        'counts' => $counts
    ]);
}
```

Důležité: AJAX endpointy také procházejí přes router a používají middleware (CSRF, Auth), ale vracejí JSON místo redirectu nebo render view.

Frontend zpracování

JavaScript v `public/js/ajax.js` zpracovává AJAX odpovědi:

```
fetch(url, {
    method: 'POST',
    body: formData
})
.then(response => response.json())
.then(data => {
    if (data.success) {
        // Aktualizace UI
    } else {
        // Zobrazení chyby
    }
});
```

Bezpečnostní aspekty

SQL Injection

Všechny dotazy používají **prepared statements**:

```
$stmt = $pdo->prepare("SELECT * FROM users WHERE id = :id");
$stmt->execute(['id' => $userId]);
```

Nikdy se nepoužívá string concatenation s uživatelskými daty.

XSS (Cross-Site Scripting)

Všechny výstupy do HTML jsou escapovány:

```
echo htmlspecialchars($userInput);
```

V šablonách: `<?= htmlspecialchars($data) ?>` nebo `<?= $data ?>` (pokud je data již sanitizováno).

CSRF

Viz sekce CSRF ochrana.

Hesla

Hesla se hashují pomocí `password_hash()`:

```
$hash = password_hash($password, PASSWORD_BCRYPT);
```

Ověření:

```
if (password_verify($password, $hash)) {
    // Heslo je správné
}
```

Upload souborů

- Validace MIME typu i přípony
- Omezení velikosti (2MB avatary, 5MB obálky)
- Ukládání relativně
- Konverze na WebP (efektivita)

Session hijacking

- `httponly` cookie (JavaScript nemůže přistupovat)
- `samesite=Strict` (ochrana proti CSRF)
- Regenerace session ID po přihlášení/odhlášení:

  ```
  session_regenerate_id(true);
  ```

Závěr

Tato aplikace demonstruje vlastní implementaci MVC architektury bez frameworku. Všechny komponenty jsou navrženy tak, aby byly srozumitelné a udržovatelné.